Complete Execution Plan

Campus Complaint & Grievance Tracker — Step-by-Step Build Guide

This document breaks the project into 10 phases, in the order you should actually do the work. Each phase explains the goal, the exact steps, the DBMS concepts involved (explained in plain language), and what you should have finished by the end of that phase. A glossary of key terms comes first so the phases read smoothly, and a set of unique features you can add is included at the end to make your project stand out beyond a basic CRUD app.

Part A — Key DBMS Terms Explained Simply

Read this once before starting. These are the terms used throughout the plan, explained in plain language rather than textbook definitions.

- **Entity:** A 'thing' you store data about — e.g. a Student, a Complaint, a Department. In a table-based database, each entity usually becomes one table.
- **Attribute:** A property of an entity — e.g. a Complaint has a description, a status, a created date. Attributes become the columns of a table.
- **Primary Key (PK):** A column (or combination of columns) that uniquely identifies each row in a table — like a roll number uniquely identifies a student. No two rows can share the same primary key.
- **Foreign Key (FK):** A column in one table that points to the primary key of another table, creating a link between them — e.g. every Complaint row stores the student's user_id so you can trace which student filed it.
- **Relationship & Cardinality:** How two entities connect, and how many of one can relate to how many of the other. '1 student files many complaints' is a one-to-many relationship.
- **ER Diagram (Entity-Relationship Diagram):** A visual diagram showing all entities, their attributes, and how they connect — the blueprint you draw before writing any SQL.
- **Generalization / Specialization:** A way of modeling 'a special type of something' — e.g. Student and Staff are both specialized types of the general entity User. It avoids repeating shared columns twice.
- **Normalization:** The process of organizing tables so that each piece of information is stored in exactly one place, avoiding duplicate or inconsistent data. Done in stages called Normal Forms.
- **Functional Dependency (FD):** A rule that says one column's value determines another's — e.g. 'student_id determines student_name' because each student_id belongs to exactly one student.
- **1NF / 2NF / 3NF / BCNF (Normal Forms):** Increasingly strict rules a table must satisfy: 1NF = no repeating groups in a cell; 2NF = no attribute depends on only part of a multi-column key; 3NF = no attribute depends on a non-key attribute instead of the key; BCNF = an even stricter version of 3NF. Each level removes a specific kind of redundancy.
- **Update / Insert / Delete Anomaly:** A data-integrity bug caused by poor table design — e.g. updating a warden's phone number in 50 different rows because it was duplicated everywhere instead of stored once.
- **DDL (Data Definition Language):** SQL commands that define structure — CREATE TABLE, ALTER TABLE, DROP TABLE. These build the database's shape.
- **DML (Data Manipulation Language):** SQL commands that work with data inside tables — INSERT, UPDATE, DELETE, SELECT.
- **Constraint:** A rule the database enforces automatically — e.g. NOT NULL (a value is required), UNIQUE (no duplicates allowed), CHECK (a value must satisfy a condition).
- **Trigger:** A small piece of code stored inside the database that runs automatically whenever a specified event happens (e.g. BEFORE an UPDATE, or AFTER an INSERT). Used here to enforce valid status transitions and to log history automatically.

- **Stored Procedure / Function:** Reusable SQL logic saved inside the database itself, which can be called by name instead of rewriting the same SQL every time — e.g. a procedure that assigns a new complaint to the least-busy staff member.
- **Event / Scheduled Job:** A trigger-like object that runs automatically on a time schedule (e.g. once a day) rather than in response to a data change. Used here for auto-escalation.
- **View:** A saved SELECT query that behaves like a virtual table. Used to show each user role only the data they're allowed to see, without duplicating logic across the app.
- **Transaction:** A group of database operations that must all succeed together, or none of them happen at all — e.g. 'assign staff AND change status to Assigned' should not be allowed to half-complete.
- **ACID Properties:** The four guarantees a good transaction gives you: Atomicity (all-or-nothing), Consistency (data always follows the rules), Isolation (concurrent transactions don't interfere with each other), Durability (once committed, it survives a crash).
- **Concurrency Control:** Techniques that let multiple users update the database at the same time without corrupting data — e.g. two staff members updating the same complaint simultaneously.
- **Index:** An internal lookup structure (similar to a book's index) that lets the database find rows fast without scanning the whole table. Speeds up SELECT queries that filter on indexed columns.
- **Query Execution Plan / EXPLAIN:** A report the database gives you showing exactly how it plans to run a query — which indexes it will use, how many rows it expects to scan. Comparing this before and after adding an index is how you prove optimization worked.
- **Query Optimization:** Rewriting a query or adding indexes so the database can answer it faster, using fewer resources.
- **Role-Based Access Control (RBAC):** Restricting what each type of user (student, staff, warden, admin) can see or do, typically enforced here through views and, optionally, database user permissions (GRANT).

Part B — Phase-by-Phase Execution Plan

Suggested total duration: 5–6 weeks. Adjust the pacing to your own deadline, but do the phases in this order — each one depends on the last.

Phase 1 — Requirement Analysis	**Days 1–2**

Goal: Decide exactly what your system needs to track before designing anything.

1. List every complaint category your campus needs: Hostel, Mess, Academics, Infrastructure/Maintenance, IT, Library, and any others relevant to your campus.
2. List the departments that handle each category (e.g. Infrastructure complaints go to the Maintenance Department).
3. Define your user roles: Student, Staff, Warden, Admin — and write one sentence per role describing what they should be able to do.
4. Decide a Service Level Agreement (SLA) — how many days a complaint of each category should reasonably take to resolve before it counts as 'stuck' (e.g. Mess: 2 days, Infrastructure: 5 days). This will directly drive your escalation logic later.
5. Write down what is explicitly out of scope (see the Synopsis document) so you don't scope-creep mid-project.

Deliverable: A one-page requirement note listing categories, departments, roles, and SLA days per category.

Phase 2 — ER Modeling	**Days 3–5**

Goal: Turn your requirements into a visual entity-relationship design.

1. List every entity you'll need: User, Student, Staff, Department, Category, Complaint, StatusHistory, Staff_Assignment.
2. For each entity, list its attributes and mark the primary key.
3. Model User as a generalized entity with Student and Staff as specialized sub-types (they share attributes like name and email but differ in details like room number vs. designation).
4. Draw the relationships between entities with correct cardinality (1:1, 1:M, M:N) — e.g. one Department handles many Categories; one Complaint has many StatusHistory rows.
5. Draw the full ER diagram using a tool such as draw.io, dbdiagram.io, or Lucidchart.

Deliverable: A complete ER diagram (image or PDF export) covering all entities and relationships.

Phase 3 — Normalization	**Days 6–7**

Goal: Prove your schema has no redundancy or update/insert/delete anomalies.

1. Write out one flat, denormalized version of your data as it would look if you dumped everything into a single spreadsheet (student name, hostel name, warden phone, category, department, status, staff name, all in one row).
2. Identify the anomalies this causes — repeat this exercise explicitly in your report (e.g. 'warden phone repeats for every complaint from that hostel — an update anomaly').
3. List the functional dependencies (e.g. complaint_id → status, created_at; category_id → department_id; hostel_id → warden_id).
4. Decompose the flat table into separate tables step by step until every table satisfies 3NF (or BCNF where applicable) — each non-key column should depend on the whole primary key, and nothing else.
5. Cross-check the final decomposition against your ER diagram from Phase 2 — they should match.

Deliverable: A short written case study (bad table → FD list → normalized tables) — this becomes a key section of your report.

Phase 4 — Physical Schema & Sample Data	**Days 8–10**

Goal: Turn the normalized design into a real, running database.

1. Choose your DBMS: MySQL or PostgreSQL (both support triggers and scheduled events, which you'll need in Phase 5).
2. Write CREATE TABLE statements (DDL) for every table, including PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints.
3. Choose sensible data types (INT for IDs, VARCHAR for short text, TEXT for descriptions, DATETIME for timestamps, ENUM for fixed status values).
4. Insert realistic sample data (DML) — at least 15–20 students, 5–8 staff, a handful of departments/categories, and 30+ sample complaints across different statuses so your demo has something to show.
5. Verify all foreign keys are enforced by trying to insert a row with an invalid reference and confirming the database rejects it.

Deliverable: schema.sql (structure) and seed_data.sql (sample data), both tested and running.

Phase 5 — Business Logic: Triggers, Procedures & Events	**Days 11–15**

Goal: Move the core rules of your system into the database itself — this is the phase that makes your project stand out.

1. Write the state-machine trigger: BEFORE UPDATE on Complaint, reject any status change that doesn't follow Open → Assigned → In-Progress → Resolved/Escalated.
2. Write the audit-logging trigger: AFTER UPDATE on Complaint, automatically insert a row into StatusHistory whenever the status actually changes.
3. Write the auto-escalation event: runs daily, escalates any complaint that has stayed in Assigned or In-Progress longer than its category's SLA (from Phase 1) — compare against the Category table's SLA column rather than a single hardcoded number, so different categories escalate at different speeds.
4. Write an auto-assignment stored procedure: when a new complaint is filed, automatically assign it to the staff member in the relevant department who currently has the fewest open complaints (a simple load-balancing rule using an aggregate subquery).
5. Test every trigger and procedure manually: try an invalid transition and confirm it's rejected; backdate a complaint's updated_at and run the escalation event manually to confirm it escalates.

Deliverable: triggers.sql and procedures.sql, each with a short test log showing they behave correctly.

Phase 6 — Views & Role-Based Access	**Days 16–17**

Goal: Make sure each user role only ever sees what they're supposed to.

1. Create vw_student_complaints — filtered to the logged-in student's own complaints only.
2. Create vw_staff_assigned — filtered to complaints currently assigned to the logged-in staff member.
3. Create vw_warden_dashboard — filtered to all complaints from the warden's hostel.
4. Create vw_admin_overview — all complaints, joined with department and category, plus a computed 'hours_open' column for analytics.
5. (Optional, for extra credit) Create actual database users with GRANT statements restricting each role to only their view — a real demonstration of database-level access control, not just application-level filtering.

Deliverable: views.sql, plus a short note on which application role maps to which view.

Phase 7 — Queries, Transactions & Optimization	**Days 18–20**

Goal: Show you can both query the data meaningfully and make those queries fast.

1. Write at least 5 'showcase' queries: one with a multi-table JOIN, one with a nested subquery, one with GROUP BY + aggregate functions (COUNT/AVG), one using HAVING, and one analytics query (e.g. average resolution time per department).
2. Wrap any multi-step operation (e.g. assign staff + update status together) inside a transaction (BEGIN ... COMMIT), and explain in your report how this guarantees atomicity — either both changes happen, or neither does.
3. Pick your most frequently run query (e.g. filtering complaints by status and category), run EXPLAIN on it, note the result, then add an index on those columns and run EXPLAIN again to show the improvement.
4. Take screenshots of both EXPLAIN outputs — this before/after comparison is one of the most convincing things you can show a professor.

Deliverable: queries.sql, and a short optimization note with the two EXPLAIN screenshots side by side.

Phase 8 — Backend API	**Days 21–25**

Goal: Build a thin server layer that simply calls your views, procedures, and triggers — no business logic duplicated here.

1. Set up a Node.js/Express or Python/Flask project and connect it to your database using a standard DB driver/connector.
2. Build a login/session mechanism that stores which user is logged in and their role.
3. Build REST endpoints: POST /complaints (create), GET /complaints (list, using the correct view based on role), PATCH /complaints/:id (update status — the trigger will reject invalid ones automatically).
4. Make sure the backend does not re-implement the state machine or escalation logic in code — let the database reject/enforce it, and simply surface the resulting error message to the frontend.

Deliverable: A running backend you can test with a tool like Postman or curl.

Phase 9 — Frontend	**Days 26–29**

Goal: Give each role a simple screen to interact with the system — this is supporting material, not the main point.

1. Build a complaint submission form (category dropdown, description box) for students.
2. Build a status-update screen for staff, and a hostel-wide table for wardens.
3. Build an admin dashboard showing counts by category/status and the average resolution time query from Phase 7, ideally as a simple chart.
4. Keep styling simple — a clean, readable interface is enough; this project is judged on the database layer.

Deliverable: A working frontend connected to your backend, covering all four roles.

Phase 10 — Testing, Report & Demo Prep	**Days 30–35**

Goal: Make sure everything works end-to-end and you can explain it confidently.

1. Test case 1: try to move a complaint directly from Open to Resolved — confirm the trigger blocks it.
2. Test case 2: manually backdate a complaint and run the escalation event — confirm it escalates and appears correctly in the admin view.
3. Test case 3: log in as two different students and confirm each only sees their own complaints.
4. Test case 4: file several complaints and confirm the auto-assignment procedure balances them across staff members.
5. Compile your final report: requirement note, ER diagram, normalization case study, schema, trigger/procedure/view scripts, sample queries with EXPLAIN screenshots, and the synopsis.
6. Rehearse a live demo script: show a normal complaint being filed and resolved, then deliberately trigger a rejected invalid transition, then show an escalation happening — in that order, since it builds from simple to impressive.

Deliverable: Final report, tested system, and a rehearsed 5-minute demo.

Part C — Unique Features to Make This Project Stand Out

Any one of these, done well, moves your project from 'another complaint form' to 'a project that actually understands databases'. You don't need all of them — pick 3–4 you can implement and explain confidently.

1. Database-Enforced State Machine

What it means: instead of trusting your website's code to only allow valid status changes, the rule lives inside the database as a trigger. Even if someone edits the database directly with a random SQL tool, bypassing your app entirely, the invalid change is still rejected. This is a strong, memorable answer to 'why is this not just a CRUD app.'

2. SLA-Driven Dynamic Escalation

What it means: instead of a single hardcoded 'escalate after 3 days for everything,' each Category stores its own SLA (e.g. Mess = 2 days, Infrastructure = 5 days), and the escalation event reads that value dynamically. This shows you understand that real systems configure business rules through data, not hardcoded logic — a subtle but impressive detail.

3. Automatic Load-Balanced Assignment

What it means: when a new complaint comes in, a stored procedure automatically assigns it to whichever staff member in that department currently has the fewest open complaints, using an aggregate COUNT() subquery. This demonstrates procedural SQL logic solving a real fairness problem, instead of leaving assignment to a manual dropdown.

4. Full Audit Trail (Temporal History)

What it means: every status change is automatically recorded — old value, new value, who changed it, and when — purely through a trigger, with zero extra code in the application. You can then show a complete timeline for any complaint, which is exactly how real-world systems (banking, healthcare records) maintain accountability.

5. Role-Based Views as Real Access Control

What it means: rather than the frontend simply 'choosing not to show' certain rows, the database itself only returns rows the role is allowed to see, through views (and optionally database GRANT permissions). This is a genuine security boundary, not just a UI convenience.

6. Recurring-Issue Detection

What it means: a query or trigger that detects when the same category of complaint keeps recurring from the same location/room/department within a short time window (e.g. 3+ water-leak complaints from the same hostel wing in 2 weeks), and flags it as a systemic issue rather than an isolated one. This is a genuinely useful, original feature — it turns your complaint log into a early-warning system for facilities management, and it's built entirely from an aggregate GROUP BY / HAVING query.

7. Query Optimization Proof

What it means: showing the actual EXPLAIN output of a query before and after adding an index, with the row-scan count visibly dropping. Most student projects skip this entirely — including it, even for just one query, immediately signals that you understand what a database engine is actually doing under the hood.

8. Analytics Views for Administration

What it means: pre-built views that answer management-level questions directly through SQL — average resolution time per department, busiest complaint category this month, staff with the most/fewest resolved complaints — turning raw transactional data into decision-ready information without any extra application code.

9. Transaction-Wrapped Multi-Step Operations

What it means: operations that touch more than one table (like assigning a staff member and simultaneously changing the status to 'Assigned') are wrapped in an explicit transaction, so they either both succeed or both fail — demonstrating the Atomicity and Consistency parts of ACID directly, rather than just mentioning ACID in your report as theory.

Suggested minimum to implement for a strong project: Features 1, 2, 4, 5, and 7 — these five give you the clearest, most demonstrable DBMS-core story without requiring extra time on things like load-balancing logic or recurring-issue analytics, which you can mention as 'future scope' if time runs short.